

Livrable 3 Générateur de données de démonstration

Livrable 3 — Générateur de données de démonstration

Découverte importante : la clé étrangère `player_match_detailed_stats.player_id` référence `joueurs`, pas `players`

Ce qui a été fait

1. Le générateur (`app/Services/RoleEvaluationDemo/`)
2. Accommodations documentées (contraintes CHECK pré-existantes)
3. Point ouvert (pas traité, faute d'accord) : élargir `match_events.type`
4. Réalisme et cas dégradés (mandat, LIVRABLE 3)
5. Marquage des données de démonstration

Ce qui a été corrigé grâce à une exécution réelle (pas seulement relue)

Validation SQLite réelle (contraintes actives, pas une relecture)

6. Tests écrits (non exécutés par PHPUnit, faute de PHP sur la machine connectée)

Comment appliquer ceci vous-même

Ce que je n'ai pas pu vérifier

Rappel des règles respectées

Prochaine étape proposée

Livrable 3 — Générateur de données de démonstration

Statut : code et migration correctrice écrits et validés (dont une exécution PHP réelle de la logique de simulation), mais rien n'est exécuté sur la base de développement. Branche `feature/role-evaluation-schema`, aucun commit encore créé à la rédaction de ce document (proposition d'en créer un, local, sans push — voir en fin de document), aucune donnée existante modifiée ou supprimée. Le jeu de démonstration actuel (49 joueurs / 10 matchs) n'a été ni supprimé ni modifié.

Ce document fait suite à `03-implementation-livrable-2.md`, que vous avez validé ("ok vas y" après arbitrage sur l'identifiant FIFA Connect et la déduplication des événements). Il couvre le Livrable 3 tel que défini au mandat : générateur reproductible, données réalistes, aucun produit cartésien, cas dégradés, et **aucune suppression du jeu actuel sans votre accord**.

Découverte importante : la clé étrangère `player_match_detailed_stats.player_id` référence `joueurs`, pas `players`

En validant ce livrable jusqu'à l'écriture SQLite réelle (voir méthodologie plus bas), une insertion de statistiques pour un joueur de démonstration a échoué avec une violation de clé étrangère. Investigation :

- `player_match_detailed_stats.player_id` référence `joueurs(id)`, pas `players(id)` — vérifié par introspection réelle (`PRAGMA foreign_key_list`) sur une copie jetable de votre base.
- Cette contrainte est **activement appliquée** : `config/database.php` a `foreign_key_constraints => env('DB_FOREIGN_KEYS', true)`, et rien dans `.env` ne la désactive. Ce n'est pas théorique.
- `joueurs` (49 lignes, id 1 à 49) **n'a aucun rapport** avec les 49 premiers id de `players`, malgré la coïncidence de plage : id=1 est "Alexandre Michel" dans `players`, "Bouazza Samir" dans `joueurs`. Le jeu de démonstration actuel (49 joueurs / 10 matchs, table `player_match_detailed_stats`) ne « fonctionne » que par cette coïncidence de plage d'identifiants — sémantiquement, ses lignes pointent vers `joueurs`, pas vers les 49 premiers joueurs de `players`.
- `joueurs` est l'ancrage d'un sous-système distinct : `player_detailed_stats`, `player_connected_devices`, `player_real_time_health`, `player_sdoh_data` référencent tous `joueurs`. Deux de ces quatre tables sont explicitement les tables de santé interdites par les RÈGLES du mandat.
- Élément rassurant : le code applicatif existant suppose **déjà** la relation vers `players`. `app/Models/Player.php` (ligne 216) déclare `hasMany(PlayerMatchDetailedStats::class, 'player_id')`. Le modèle `Joueur.php` ne déclare aucune relation vers `player_match_detailed_stats`. La contrainte en base contredit donc le code applicatif lui-même.

Je vous ai soumis trois options ; vous avez choisi : **préparer la migration correctrice sans l'exécuter**. C'est fait — voir `database/migrations/2026_09_30_110000_fix_player_match_detailed_stats_player_foreign_key.php`, non exécutée, qui répare uniquement cette clé étrangère (`joueurs` → `players`), sans toucher `joueurs` elle-même ni les quatre autres tables qui la référencent (hors périmètre des trois livrables, et deux d'entre elles sont des tables de santé interdites).

Cette migration est un prérequis du Livrable 3 : sans elle, l'écriture de `player_match_detailed_stats` pour des joueurs de démonstration échoue avec une violation de clé étrangère. Elle doit être appliquée **après** les 12 migrations du Livrable 1 et **avant** `php artisan role-eval:generate-demo` (voir "Comment appliquer" en fin de document). Elle répond directement à la question posée au Livrable 1 §9 ("table de référence unique pour les joueurs : recommande entre players et joueurs") — avec, cette fois, une preuve concrète et exécutable de l'ambiguïté, pas seulement un signalement.

Ce qui a été fait

1. Le générateur (`app/Services/RoleEvaluationDemo/`)

Six classes, dont quatre n'ont **aucune dépendance à Laravel** (ce qui a permis de les exécuter réellement, voir validation) :

- `SeededRandom` — générateur pseudo-aléatoire déterministe. Premier essai basé sur `mt_srand()` / `mt_rand()` de PHP : **invalidé par l'exécution réelle** (voir "Ce qui a été corrigé"), remplacé par une implémentation auto-contenue (xorshift32) dont l'état vit entièrement dans l'objet, sans dépendre de l'état global du processus PHP.
- `PositionCatalog` — constantes dupliquant le référentiel des 12 postes détaillés (Livrable 1 §9), le repli vers le vocabulaire restreint existant (`position_played` , 10 codes), et le gabarit d'effectif (~20 joueurs/équipe).
- `ScheduleGenerator` — calendrier en double round-robin (méthode du cercle). Avec 16 clubs, donne exactement $2 \times (16-1) = 30$ **matchs par équipe**, comme demandé ("au moins 30").
- `DemoSquadFactory` — construit en mémoire des clubs, équipes et joueurs fictifs (nom + identifiant FIFA Connect "DEMO-CLUB-xxx" clairement marqués — voir "Marquage des données de démonstration").
- `MatchStatSimulator` — simule un match complet : composition (11 titulaires + remplaçants), minutes cohérentes, événements (buts/cartons/remplacements), statistiques joueur brutes dépendantes du poste (niveau propre au joueur + bruit), statistiques d'équipe **agrégées** depuis les statistiques joueurs, score **dérivé** des événements de but.
- `DemoDataGenerator` — orchestrateur : seul composant qui parle à la base (`DB::table(...)->insert(...)` , jamais `update()` / `delete()` sur une table existante), transaction + dry-run (même mécanique que le Livrable 2 : rollback via `DryRunRollback`), lot d'import (`import_batches` , `batch_type='demo_generation'`).

Et la commande : `php artisan role-eval:generate-demo --seed=42 --clubs=16 [--dry-run]` .

2. Accommodations documentées (contraintes CHECK pré-existantes)

Comme au Livrable 2 (`position_played`), plusieurs contraintes CHECK déjà en base ont dû être respectées **sans être modifiées** :

- `match_events.type` est restreint à (`'goal','yellow_card','red_card','substitution','injury'`) — sans `own_goal` , `second_yellow_card` ni `pass` . Conséquences :
 - un but contre son camp est enregistré `type='goal'` , `team_id` = l'équipe **bénéficiaire**, avec `event_data = {"own_goal": true}` — jamais crédité au buteur dans ses statistiques (comme en football réel) ;
 - un joueur ne reçoit jamais plus d'un carton par match (pas de "deuxième jaune") ;
 - aucun événement de passe n'est généré** — le mandat liste "buts, cartons, remplacements, puis passes avec destinataire et zone" ; les trois premiers sont couverts, le quatrième nécessiterait d'élargir ce CHECK (voir "Point ouvert" ci-dessous).
- `player_match_detailed_stats.data_source` est restreint à 6 valeurs, sans option "démon" : `manual_entry` est utilisé (le moins trompeur des six).
- `player_match_detailed_stats.data_quality` est restreint à `excellent/good/fair/poor` : `fair` est utilisé.
- `player_match_detailed_stats.position_played` garde le vocabulaire à 10 codes (Livrable 2) : correspondance many-to-one depuis les 12 codes détaillés (ex. LCB et RCB donnent tous les deux "CB").

Dans tous les cas, `is_demo=true` reste le signal faisant foi pour l'interface ; ces choix de valeurs n'ont d'autre but que de respecter des contraintes SQL déjà en place.

3. Point ouvert (pas traité, faute d'accord) : élargir `match_events.type`

Pour générer un jour des passes avec destinataire/zone (colonnes déjà ajoutées au Livrable 1 : `recipient_player_id` , `origin_zone` , `destination_zone`), il faudrait élargir le CHECK de `match_events.type` pour y ajouter `'pass'` (et, tant qu'à faire, `'own_goal'` et `'second_yellow_card'` pour des raisons similaires). C'est une **modification d'une contrainte existante**, donc hors de portée sans votre accord écrit — je ne l'ai ni proposée en migration ni exécutée. Dites-moi si vous voulez que je prépare cette migration séparément.

4. Réalisme et cas dégradés (mandat, LIVRABLE 3)

Exigence du mandat	Implémentation
Plusieurs équipes, effectifs ~20 joueurs	16 clubs fictifs, gabarit exact de 20 joueurs/équipe (<code>PositionCatalog::SQUAD_TEMPLATE</code>)
11 titulaires + remplaçants	Formation 4-3-3 fixe (<code>STARTING_ELEVEN_SLOTS</code>), 0 à 4

	remplacements/match/équipe
Minutes cohérentes avec entrées/sorties	<code>minute_out</code> d'un titulaire = minute de sortie s'il est remplacé, sinon 90 ; <code>minute_in</code> d'un remplaçant = sa minute d'entrée ; <code>minutes_played</code> toujours > 0
Statistiques dépendantes du poste, niveau + bruit	Profil par poste détaillé (12) × niveau du joueur [0,1] dérivé de la graine (non persisté) × bruit gaussien par match
≥ 30 matchs par équipe	Double round-robin, 16 clubs → exactement 30
Stats équipe cohérentes avec la somme des stats joueurs	Agrégation directe (<code>array_sum</code>) — garanti par construction, pas par contrôle a posteriori
Événements cohérents avec le score	Score = <code>COUNT</code> des événements de but déjà générés — garanti par construction
Aucun produit cartésien	~26 participations par match (deux équipes), jamais (joueurs × matchs)
<code>is_demo = vrai</code> partout	<code>clubs</code> / <code>competitions</code> / <code>match_sheets</code> n'ont pas de colonne <code>is_demo</code> (Livrable 1 ne l'y a pas ajoutée) — identifiables par jointure vers un match/équipe/joueur <code>is_demo=true</code> , ou par convention de nom ("(Démo)", "DEMO-CLUB-xxx")
Cas dégradés	Voir ci-dessous

Trois cas dégradés, choisis **déterministement** par index de calendrier (pas par tirage aléatoire supplémentaire), pour rester repérables d'une exécution à l'autre avec la même graine :

- Indicateurs manquants** : sur les matchs d'indices 0, 50, 100, 150, 200 du calendrier généré, plusieurs colonnes nullables (buts attendus, chances créées, passes progressives...) sont mises à `NULL` plutôt que calculées — jamais 0, conformément à la règle du mandat.
- Joueur à très peu de minutes** : sur les matchs d'indices 25, 75, 125, 175, 225, le dernier remplacement de la rencontre est forcé entre la 86e et la 89e minute (1 à 4 minutes jouées).
- Changement de poste en cours de saison** : un joueur du club d'indice 0 (initialement CDM) devient LCB à partir de la seconde moitié du calendrier (journée 16 sur 30) — visible en comparant `match_participations.detailed_position` pour ce joueur avant/après.

5. Marquage des données de démonstration

- `matches`, `players`, `teams`, `match_participations`, `player_match_detailed_stats`, `match_team_stats`, `match_events` : `is_demo=true` + `import_batch_id` (Livrable 1 §7).
- `clubs` et `competitions` n'ont **pas** de colonne `is_demo` (le Livrable 1 ne l'y a pas ajoutée — seuls `players` / `teams` / `matches` l'ont reçue, en plus des tables de données de match). Je les ai donc marqués par convention : nom suffixé "(Démo)", identifiant FIFA Connect fictif au format `DEMO-CLUB-xxx`. Si vous préférez une colonne `is_demo` sur ces deux tables aussi, c'est une migration additionnelle simple à proposer séparément.
- `match_sheets` n'a pas non plus de colonne `is_demo` — identifiable uniquement par jointure vers `matches.is_demo`.

Ce qui a été corrigé grâce à une exécution réelle (pas seulement relue)

Cet environnement cloud dispose de PHP 8.4 (contrairement à la machine connectée, où `php` reste introuvable) — sans Laravel/Composer installés pour autant. Cela m'a permis d'exécuter **réellement** les quatre classes sans dépendance framework (`SeededRandom`, `PositionCatalog`, `ScheduleGenerator`, `DemoSquadFactory`, `MatchStatSimulator`) en dehors de tout artisan/PHPUnit, et d'y trouver deux défauts réels :

- `SeededRandom` n'était pas réellement reproductible entre deux instances.** Le premier essai enveloppait `mt_srand()` / `mt_rand()` de PHP, qui reposent sur un état **global au processus**. Deux instances construites dans le même processus se marchaient dessus dès que leurs appels s'entrelaçaient (le second constructeur réinitialisait l'état utilisé par le premier objet). Détecté par un test réel (deux instances, même graine, séquences comparées) qui a **échoué à l'exécution**. Corrigé en remplaçant l'implémentation par un xorshift32 auto-contenu (l'état vit dans l'objet, pas dans le processus).
- Un remplaçant pouvait entrer à la 90e minute exacte**, jouant 0 minute — plus une participation valide. Détecté par un test réel sur les 240 matchs d'une saison complète simulée. Corrigé en bornant la fenêtre du cas dégradé "peu de minutes" à la 86e-89e minute.

Après ces deux corrections, un harnais de validation a **réellement simulé 240 matchs** (une saison complète à 16 clubs) et vérifié, sur les données réellement produites (pas une relecture du code) :

- reproductibilité vérifiée **entre deux processus PHP séparés** (pas seulement deux objets dans le même processus) : `--seed=2026` donne le même hachage SHA-1 sur l'intégralité des scores et statistiques de 240 matchs à deux exécutions distinctes ; `--seed=777` donne un hachage différent ;

- score toujours égal au nombre d'événements de but générés, pour les 240 matchs ;
- aucun événement en dehors du vocabulaire CHECK existant ;
- statistiques d'équipe = somme exacte des statistiques joueurs (tirs) ; buts d'équipe $\geq$ somme des buts joueurs (l'écart, quand il existe, correspond exactement aux buts contre son camp, jamais crédités à un joueur) ;
- réussites $\leq$ tentatives sur passes/tacles/dribbles/tirs/arrêts, sur l'ensemble des 240 matchs ;
- toute participation écrite a `minutes_played > 0` ;
- les deux cas dégradés "indicateurs manquants" et "peu de minutes" se déclenchent bien ; au moins un but contre son camp observé sur l'échantillon (probabiliste, 2 % par tentative de but).

Validation SQLite réelle (contraintes actives, pas une relecture)

Au-delà de la logique pure, j'ai fait produire par PHP la sortie **réelle** du simulateur pour deux matchs (un normal, un avec tous les cas dégradés activés), puis inséré ces données — avec exactement le mapping de colonnes utilisé par `DemoDataGenerator` — dans une **copie jetable** de votre base de développement, `PRAGMA foreign_keys=0` (comme en production), après reconstruction du DDL des Livrables 1 et de la migration correctrice de ce livrable. C'est cette validation qui a révélé la découverte sur `joueurs / players` (voir plus haut). Après application de la migration correctrice :

- 0 erreur d'insertion (CHECK/FK/NOT NULL) sur l'ensemble des tables touchées ;
- `PRAGMA foreign_key_check` : aucune anomalie ; `PRAGMA integrity_check` : ok ;
- le jeu existant est resté strictement identique : 825 joueurs non-démo avant/après, les 34 lignes de `matches` 1-34 byte pour byte identiques, 490 lignes de `player_match_detailed_stats` pour le club CS Sfaxien avant/après ;
- aucune valeur de `match_events.type` ni de `position_played` en dehors des CHECK existants ;
- `score = COUNT(match_events, type='goal')` pour tous les matchs générés.

Copie jetable et fichiers de travail supprimés après usage, comme pour les Livrables 1 et 2.

6. Tests écrits (non exécutés par PHPUnit, faute de PHP sur la machine connectée)

- `tests/Unit/RoleEvaluationDemo/SeededRandomTest.php` — reproductibilité, bornes, permutation.
- `tests/Unit/RoleEvaluationDemo/ScheduleGeneratorTest.php` — 30 matchs/équipe exactement sur 16 clubs, aucune équipe contre elle-même, aucun couple domicile/extérieur en double, cas d'un nombre impair d'équipes (bye).
- `tests/Feature/GenerateDemoDataCommandTest.php` — `DatabaseTransactions` (jamais `RefreshDatabase`, même règle qu'aux Livrables 1 et 2 : `phpunit.xml` pointe sur la base de développement elle-même) : dry-run ne persiste rien ; toutes les lignes générées portent `is_demo=1` ; volumétrie sans produit cartésien ; score cohérent avec les événements ; stats d'équipe = somme des stats joueurs ; **le jeu de démonstration actuel (club CS Sfaxien, matchs 1-34) n'est jamais modifié.**

Ces tests couvrent exactement les propriétés déjà vérifiées réellement ci-dessus (SeededRandom/ScheduleGenerator en PHP pur, le reste par la validation SQLite) — ils sont donc écrits avec une forte confiance qu'ils passeront, mais je n'ai pas pu les faire tourner sous PHPUnit avec Eloquent/Laravel chargés.

Comment appliquer ceci vous-même

1. **Sauvegarde** de `database/database.sqlite` avant toute manipulation.
2. `php artisan migrate` — applique les 12 migrations du Livrable 1 **et** la migration correctrice de ce livrable (`2026_09_30_110000_...`), dans cet ordre (le préfixe de date le garantit).
3. `php artisan test --filter=RoleEvaluationSchemaTest` (Livrable 1), puis les tests d'import (Livrable 2), puis : `php artisan test tests/Unit/RoleEvaluationDemo tests/Feature/GenerateDemoDataCommandTest.php`
4. Essai à blanc d'abord : `php artisan role-eval:generate-demo --seed=42 --dry-run` — vérifie le rapport affiché sans rien écrire.
5. Génération réelle : `php artisan role-eval:generate-demo --seed=42` (16 clubs par défaut, 240 matchs, ~30 minutes-hommes de calcul réparties sur ~6 700 participations — pas mesuré en conditions réelles, PHP absent de la machine connectée).
6. En cas de problème : la génération est une seule transaction — un échec en cours de route ne laisse rien en base (rollback automatique Laravel sur exception).

Ce que je n'ai pas pu vérifier

- **Exécution réelle de `php artisan migrate` et de la commande `role-eval:generate-demo`** dans Laravel : aucun PHP sur la machine connectée. La validation ci-dessus est réelle pour la logique pure (exécution PHP effective) et pour la conformité SQL (insertion réelle sous contraintes actives), mais pas pour l'orchestration Laravel elle-même (`DB::transaction()`, `Schema::hasTable()`, l'Artisan command, Eloquent).
- **Le comportement exact de `dropForeign()` / `foreign()` de Laravel sur SQLite** pour la migration correctrice : elle s'appuie sur `doctrine/dbal` (présent, `^3.10`) pour reconstruire la table, ce qui est la voie normale sur SQLite pour changer

une contrainte — mais non exécutée ici, donc non vérifiée en conditions réelles. J’ai simulé la même reconstruction de table directement en SQL sur la copie jetable (validée), qui suit le même principe.

- **Le temps réel de génération** de 240 matchs sous Laravel/Eloquent (probablement plus lent que les `DB::table()->insert()` bruts utilisés ici, mais l’implémentation n’utilise déjà qu’Eloquent-free `DB::table()`, pas de modèles Eloquent, pour limiter ce risque).
- **Si le Livrable 2 (l’importeur) est affecté par le même problème de clé étrangère** `joueurs` / `players` : probable (même table cible, `player_match_detailed_stats`), mais je ne l’ai pas revérifié ici — à confirmer avec vous dès qu’un accès PHP réel est disponible. Les exemples de mapping du Livrable 2 utilisaient des joueurs créés en base de test, potentiellement avec des id hors de la plage 1-49 : à re-tester une fois la migration correctrice appliquée.
- **Le modèle statistique lui-même n’a aucune prétention scientifique** : c’est un générateur de démonstration structurellement cohérent (bornes, sommes, CHECK respectés), pas une simulation calibrée sur des données réelles — cohérent avec la règle du mandat (“les données de démonstration ne servent jamais à régler des poids... ces réglages attendent les vraies données”).
- **Portabilité 32 bits de `SeededRandom`** : les opérations sont masquées sur 32 bits (`& 0xFFFFFFFF`), ce qui suppose des entiers PHP 64 bits en interne (le cas sur toute installation PHP moderne, dont celle testée ici) — non testé sur une build PHP 32 bits.

Rappel des règles respectées

- Aucune table ni donnée existante modifiée ou supprimée : uniquement des `INSERT` dans `DemoDataGenerator` (aucun `update()` / `delete()` / `truncate()` sur une table de données existante — seule exception : la mise à jour du **propre** `import_batches` que le run vient de créer, pour y inscrire son statut final).
- Le jeu de démonstration actuel (49 joueurs / 10 matchs) n’a été ni supprimé ni modifié — vérifié réellement (contenu byte-à-byte identique avant/après sur la copie de validation).
- Aucune donnée médicale/santé touchée — et la découverte sur `joueurs` renforce cette discipline : je n’ai pas écrit dans `joueurs` ni dans les tables de santé qui s’y rattachent pour contourner le problème de clé étrangère.
- Aucune donnée nominative réelle dans le code, les tests ou ce document (noms de joueurs/clubs fictifs, générés).
- Aucun poids/seuil de `role_config_weights` réglé par ce livrable.
- Le cockpit et l’onglet “Statistiques avancées” n’ont pas été touchés.
- Travail sur la branche `feature/role-evaluation-schema`, rien poussé.

Prochaine étape proposée

Un commit **local** regroupant le générateur, la migration correctrice, les tests et cette documentation — toujours sans push. Dites-moi si je le crée.

Une fois un accès PHP disponible (ici ou sur votre machine), l’ordre d’exécution recommandé est : migrations du Livrable 1 → migration correctrice de ce livrable → tests des trois livrables → `role-eval:generate-demo --dry-run` → génération réelle. Je reste disponible pour vérifier le résultat avec vous à ce moment-là, et pour traiter les points laissés ouverts (passes avec destinataire/zone, `is_demo` sur `clubs` / `competitions`, sort du jeu de démonstration actuel).